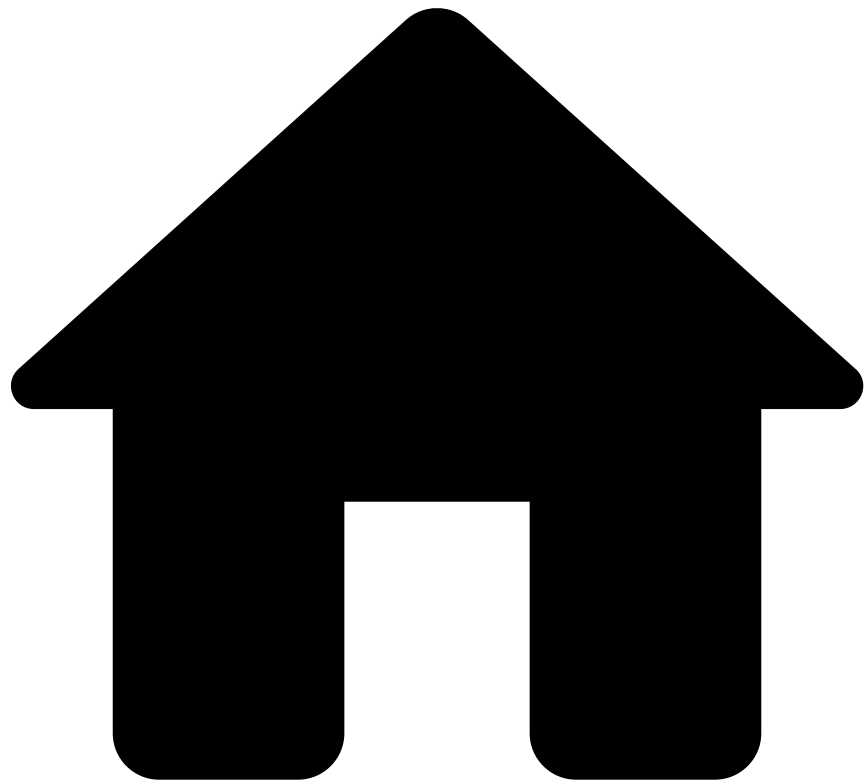


•



- [Deploying a project into Production](#)
- [Designing a workflow](#)
- Advanced workflow configurations

On this page

Advanced workflow configurations

Was this page helpful? 

The following section explains the various configurations that you can set in the Advanced tab of a specific workflow (as seen in the following image):

Action parameters

Each action can have its own parameters. Parameters allow passing event specific information when an action is triggered.

One example would be for a "sendEmail" action to have the parameters "to" and "title" with values "" and "Rule Alert".

In the **Workflow Advanced** tab, it is possible to define which parameters each action has (by default has none) in the configure action dialog (accessible by clicking the config button on the action).

When defining parameters it is possible to choose their name, type and if its value is required or optional.

Parameters can have simple types like *String* or *Number* or more complex ones like:

- **Dimension**, restrict the set of possible values of the parameter to a Dimension's items.
- **List**, the value of the parameter will be a list (not a list item). This can be useful to reference, for example, card blacklists.
- **RDL Variable**, when configuring the value of the parameter it will be possible to choose from the list of RDL variables available within the rule. When the action is triggered the parameter will have the value of that variable.

Currently, actions can only be triggered directly by [rules](#) where it is possible to specify the value that each action parameter will have when the action is triggered. This can be done by clicking the config button available on the action that has more than one parameter.

Action listeners🔗🔗

When configuring action listeners, the list of available listeners is populated from the existing services. For a service to be listed either:

- The service is a part of a workflow
- The service is part of the overall application environment and implements a `com.feedzai.pulse.api.apps.model.rte.workflow.action.WorkflowActionHandler` interface from the `com.feedzai.pulse:pulse-api` module

Event Storage configuration🔗🔗

This option allows to define whether an Event Storage is created for the workflow. The recommendation is to enable this as it is a requirement for most workflows (it is necessary for Workflow recovery, Scheduled Metrics Jobs as well as feedback updates).

Workflow recovery configuration🔗🔗

This allows to define whether Workflow recovery is enabled. Most workflows should enable it and define the recovery period to be the same as the largest real-time metric (typically 24h or less).

When recovery is enabled, you must also provide the recovery time configuration in the *Interval* field, which will correspond to the time frame we would like to replay events from. Currently, two functions can be used to determine the interval of the data that is going to be recovered: **sinceLast** and **last**.

The **sinceLast** function will recover all events since the specified last point in time. The **last** function will recover all events from the specified past point in time to the last timestamp in the event storage.

The syntax for these functions is the following:

- `sinceLast-HHhMMm`
 - HH - The hours in 24-hour mode (0-23). This field is **optional**.
 - MM - The minutes (0-59). This field is **mandatory**.
- `last-HHhMMm`
 - HH - The hours. We are not restricted by the values 0-23. This field is **optional**.
 - MM - The minutes. We are not restricted by the values 0-59. This field is **mandatory**.

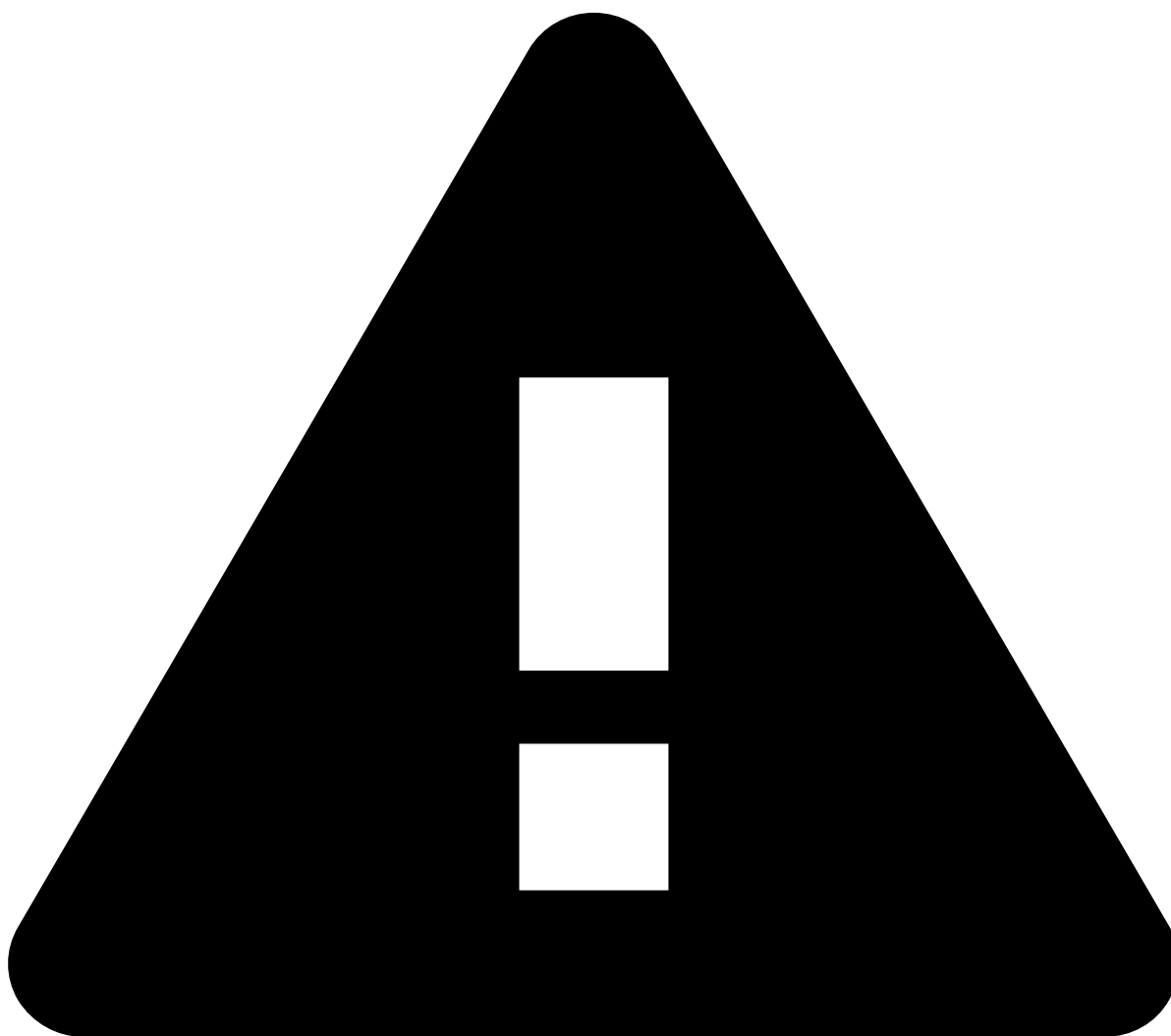
Below are some examples of recovery configurations:

- **last-30m** - With this configuration, if the last event in the event storage is at 16h30, it will recover events from 16h00. If the last event is at 00h10, it will recover events since 23h40 from the previous day. In other words, it always recovers the last 30 minutes counting back from the time of the last event in the event storage.
- **last-48h0m** - Recovers the last two days of events.
- **sinceLast-23h30m** - Using this configuration, if the last event is at 23h31, it will recover events from 23h30 in the present day (in this case 1 minute). If the last event is at 22h00, then it will recover events since 23h30 from the previous day, i.e. the last 22 hours and 30 minutes.

The default value is *last-30m*.

Advanced properties: Checkpoint-based workflow recovery [🔗](#)

In a production environment, Pulse can be configured to store the necessary events for real-time metrics calculation on disk. In this case, Pulse creates **checkpoints**, i.e. regular snapshots about the events and the state of the metrics. After a server failure, Pulse can reload the last checkpoint and recover the metrics status starting from there. This saves time when the server needs to be restarted. However, it takes toll on the metrics calculation speed, as Pulse puts metrics calculation on hold while creating a checkpoint. Hence, this setting needs to be carefully tested.



Limitations

Note that this feature is in continuous development, and as of now it does not provide full feature parity to the previous, RAM-based version.

Additionally, if this feature is enabled, do not edit or remove existing metrics. Edited metrics are not recoverable after a server failure, meaning that those metrics will only take new events into consideration after any server restart.

Partition Keys

Allows to configure the partitioning keys for scaling horizontally the workflow processing across Pulse replicas.

Other properties

We also allow to add arbitrary key/value properties to the workflow advanced configuration. The possible properties that are considered here are the ones under the *advanced* portion in Workflow Manager Service Configuration.



info

We recommend using Pulse properties

to manage those rather than defining them in the workflow, as doing so makes the properties bound to the application metadata when most of those properties are environment-specific (i.e., they can change from production to development).

Workflow slug

The only property that must be configured in the workflow advanced properties is the workflow slug. This allows to then define different Workflow Manager Service Configuration properties for different workflows (with various slugs) across the same application scoped prefix pulse property.

You can do so by setting the key to *workflowSlug* and the value to the intended slug. Then use that slug where mentioned in the Workflow Manager page.